

IntellCar

Proyecto Intermodular CFGS Desarrollo Aplicaciones Web

- José Manuel Villanúa Pozo
- Juan Benítez Munoz

Centro Público Integrado de Formación Profesional
Alan Turing

Málaga Tech Park,
15 de Mayo de 2026

Sumario

0 - Preludio / Prelude

- Abstract & Keywords
- Glosario de términos

1 - Introducción y Objetivos

- 1.1 - Resumen del proyecto
- 1.2 - Motivación
- 1.3 - Objetivos

2 - Análisis del Sistema de Software

- 2.1 - Requisitos
 - 2.1.1 - Funcionales
 - 2.1.2 - No funcionales
- 2.2 - Resumen de Viabilidad
- 2.3 - Casos de uso)

3 - Diseño de la Solución

- 3.1 - Estructura general del sistema y Arquitectura
- 3.2 - Diseño BBDD
- 3.3 - Diseño de la API
- 3.4 - Identidad visual

4 - Desarrollo técnico (Core)

- 4.1 - Backend (Laravel)
 - 4.1.1 - Autenticación con Sanctum.
 - 4.1.2 - Integración de AWS SDK (Rekognition y Comprehend).
- 4.2 - Frontend (Turborepo)
 - 4.2.1 - Estructura del Monorepo (pnpm/ Turbo)
 - 4.2.2 - Experiencia de usuario con GSAP y Motion
- 4.3 - Infraestructura como Código con Terraform (IaC):
 - 4.3.1 - ¿Por qué Terraform?
 - 4.3.2 - VPC
 - 4.3.3 - EC2
 - 4.3.4 - RDS
 - 4.3.5 - S3
 - 4.3.6 - Elastic IP
 - 4.3.7 - Security Groups

5 - Seguridad, Calidad y Despliegue

- 5.1 - Moderación automática por IA
- 5.2 - Dockerización en entornos locales en desarrollo y en AWS en producción.
- 5.3 - Despliegue continuo (Deploy.yml con GitHub Actions)
- 5.4 - Agregación del Dominio Duck DNS

5.5 - Certificación SSL

6 - Conclusiones.

6.1 - Conclusiones Técnicas

6.2 - Conclusiones Personales (José)

6.3 - Conclusiones Personales (Juan)

7 - Futura Expansión del proyecto

7.1 - App en React Native

7.2 - Pasarela de pago (Stripe)

7.3 - Versión PRO

8 - Anexos

8.1 - Plan de Empresa Completo (IPE II).

Abstract

IntellCar is a full-stack web platform designed for motor enthusiasts, combining four core features in a single ecosystem: a vehicle marketplace, a community events system (*Kdds*), a social network feed (*Universe*), and a personal virtual garage (*My Garage*). The backend is built with **Laravel 12** as a RESTful API authenticated via **Laravel Sanctum** (Bearer token), persisting data in a relational **MySQL** database with 21 domain tables. The frontend is a **Single Page Application (SPA)** developed in **Angular 19** using standalone components, organized as a **Turborepo monorepo** with **pnpm** workspaces. The infrastructure is containerized with **Docker/Docker Compose** and provisioned on **AWS** through **Terraform** scripts, enabling reproducible and scalable deployments. The API is fully documented with **Swagger/OpenAPI 3.0** through the L5-Swagger integration.

Keywords

Laravel 12 · Angular 19 · REST API · Laravel Sanctum · MySQL · Monorepo · Turborepo · pnpm · Docker · Terraform · GCP · Swagger / OpenAPI · Marketplace · Red social · Comunidad automovilística · SPA · TypeScript · PHP

Glosario de Términos

Término	Definición
API REST	Interfaz de comunicación entre cliente y servidor que sigue los principios arquitectónicos REST (stateless, recursos, verbos HTTP).
Bearer Token	Mecanismo de autenticación basado en un token opaco enviado en la cabecera HTTP <code>Authorization</code> .
CRUD	Acrónimo de las cuatro operaciones básicas sobre datos: Create, Read, Update, Delete.
Docker	Plataforma de contenedores que empaqueta una aplicación junto a sus dependencias en unidades aisladas y reproducibles.
Eloquent	ORM (Object-Relational Mapping) de Laravel que abstrae las consultas SQL mediante modelos PHP.
AWS	Amazon Web Services. Proveedor de servicios cloud donde se despliega IntellCar en producción.

Kdd	Abreviatura coloquial de "quedada". En IntellCar representa un evento automovilístico organizado por usuarios.
Laravel Sanctum	Paquete oficial de Laravel para la emisión y validación de tokens de API ligeros (Bearer).
Monorepo	Estrategia de organización de código en la que múltiples proyectos conviven en un único repositorio.
Paddock	Comunidad o estilo de vida del motor al que puede pertenecer un usuario (ej. clásicos, tuning, offroad...). En automovilismo, el paddock es la zona reservada de los equipos.
pnpm	Gestor de paquetes Node.js de alto rendimiento, usado aquí para gestionar el monorepo del frontend.
SPA	Single Page Application. Aplicación web que carga una sola vez y actualiza el contenido dinámicamente sin recargar la página.
Seeder	Script de Laravel que inserta datos de prueba en la base de datos para facilitar el desarrollo y los tests.
Soft Delete	Borrado lógico: el registro no se elimina físicamente, sino que se marca como eliminado mediante un campo de fecha (<code>onDeleteRequest</code>).
Swagger / OpenAPI	Estándar de descripción de APIs REST. Swagger UI genera una interfaz interactiva a partir de esa descripción.
Terraform	Herramienta de Infraestructura como Código (IaC) que permite definir y aprovisionar recursos cloud de forma declarativa.
Turborepo	Sistema de builds de alto rendimiento para monorepos JavaScript/TypeScript. Gestiona el grafo de dependencias entre paquetes.

1 — Introducción y Objetivos

1.1 Resumen del Proyecto

IntellCar es una plataforma web de comunidad automovilística desarrollada como **Proyecto Intermodular del ciclo formativo de Desarrollo de Aplicaciones Web (DAW)**, curso 2025/2026. El proyecto integra los conocimientos adquiridos a lo largo del ciclo en un producto full-stack funcional y desplegable.

La plataforma se articula en cuatro módulos principales o "distritos":

1. **Marketplace** — Sección de compraventa de vehículos con anuncios categorizados, filtros avanzados (marca, potencia, estilo de vida) y contacto directo con el vendedor.
2. **Eventos (Kdds)** — Sistema de quedadas y eventos ,gestión de aforo y asistencia.
3. **El Universo** — Feed de red social donde los usuarios publican artículos, fotos de modificaciones, noticias del sector y rutas. Incluye sistema de likes, comentarios y seguimiento entre usuarios.
4. **Mi Garaje** — Espacio personal donde cada usuario registra su historial de vehículos (actuales y pasados), personalizándolos con apodos, fotos e historias.

Tecnológicamente, el sistema se divide en:

- **Backend:** API REST en Laravel 12 con base de datos MySQL y autenticación Sanctum.
 - **Frontend:** SPA en Angular 19, organizada en un monorepo con Turborepo y pnpm.
 - **Infraestructura:** Contenedores Docker y despliegue cloud en AWS mediante Terraform.
-

1.2 Motivación

La afición al motor es uno de los hobbies con mayor arraigo en España, con millones de propietarios de vehículos, aficionados al tuning, coleccionistas de clásicos y participantes en eventos de velocidad. Sin embargo, el ecosistema digital disponible para esta comunidad presenta importantes carencias:

- Las plataformas de compraventa generalistas (Wallapop, Milanuncios, Coches.net) no ofrecen contexto comunitario ni segmentación por estilo de vida del motor.
- Las redes sociales genéricas (Instagram, Facebook) carecen de funcionalidades específicas para el sector: ficha técnica de vehículos, organización de quedadas, garaje virtual.
- No existe ninguna plataforma que **unifique en un único ecosistema** la compraventa, la socialización, los eventos y la gestión del historial de vehículos propios.

Adicionalmente, desde el punto de vista formativo, el proyecto representa una oportunidad ideal para aplicar en un contexto real las tecnologías impartidas en el ciclo: arquitectura cliente-servidor, diseño de bases de datos relacionales, desarrollo de APIs REST, frameworks modernos de frontend, autenticación, seguridad, despliegue en contenedores y aprovisionamiento de infraestructura AWS.

1.3 Objetivos

Objetivos Generales

- Desarrollar una plataforma web full-stack funcional orientada a la comunidad del motor, que sirva como demostración integral de las competencias adquiridas durante el ciclo DAW.
- Producir un software de calidad, documentado, testado y desplegable en entornos reales.

Objetivos Específicos

ID	Objetivo
OBJ-01	Diseñar e implementar una base de datos relacional MySQL con más de 20 tablas que modele el dominio completo de IntellCar.
OBJ-02	Desarrollar una API REST con Laravel 12 que exponga todos los recursos del sistema con separación de rutas públicas y protegidas.
OBJ-03	Implementar un sistema de autenticación seguro basado en Laravel Sanctum con tokens Bearer.
OBJ-04	Construir un SPA con Angular 19 (standalone components) que consuma la API y ofrezca una experiencia de usuario fluida.
OBJ-05	Organizar el frontend en una arquitectura monorepo (Turborepo + pnpm) que facilite la escalabilidad y reutilización de componentes.
OBJ-06	Documentar la API con Swagger/OpenAPI 3.0 mediante L5-Swagger, generando una interfaz interactiva.
OBJ-07	Implementar el módulo Marketplace con publicación, búsqueda, filtrado y gestión de anuncios de vehículos.
OBJ-08	Implementar el módulo social (El Universo) con posts, likes, comentarios y sistema de seguimiento.
OBJ-09	Implementar el módulo de eventos (Kdds) con geolocalización y gestión de asistencia.
OBJ-10	Implementar el Garaje Virtual con historial de vehículos por usuario.

OBJ-11	Contenerizar la aplicación completa con Docker y Docker Compose para garantizar entornos reproducibles.
OBJ-12	Provisionar la infraestructura de producción en AWS mediante Terraform.

2 — Análisis del Sistema de Software

2.1 Requisitos

2.1.1 Requisitos Funcionales

ID	Requisito	Módulo
RF-01	El sistema permitirá el registro de nuevos usuarios con nombre, email, teléfono, contraseña y paddock.	Autenticación
RF-02	El sistema permitirá el inicio de sesión mediante email y contraseña, devolviendo un token Bearer.	Autenticación
RF-03	El sistema permitirá el cierre de sesión invalidando el token activo.	Autenticación
RF-04	Un usuario autenticado podrá consultar y editar sus datos de perfil (nombre, teléfono, email de contacto, foto).	Perfil
RF-05	Un usuario podrá solicitar la eliminación lógica (soft delete) de su propia cuenta.	Perfil
RF-06	Un usuario autenticado podrá añadir, editar y eliminar vehículos de su garaje virtual.	Garaje
RF-07	Cualquier visitante podrá consultar el listado de anuncios del marketplace con filtros.	Marketplace
RF-08	Cualquier visitante podrá ver el detalle de un anuncio de vehículo.	Marketplace

RF-09	Un usuario autenticado podrá crear, editar y solicitar el borrado de sus propios anuncios.	Marketplace
RF-10	Un administrador podrá eliminar definitivamente cualquier anuncio.	Marketplace
RF-11	Cualquier visitante podrá consultar el feed de posts del Universo.	Social
RF-12	Un usuario autenticado podrá crear posts con texto y multimedia.	Social
RF-13	Un usuario autenticado podrá dar/quitar "like" a cualquier post.	Social
RF-14	Un usuario autenticado podrá comentar posts.	Social
RF-15	Un usuario autenticado podrá seguir y dejar de seguir a otros usuarios.	Social
RF-16	Cualquier visitante podrá consultar el listado de eventos (Kdds).	Eventos
RF-17	Un usuario autenticado podrá crear, editar y eliminar sus propios eventos.	Eventos
RF-18	Un usuario autenticado podrá inscribirse y desinscribirse en eventos.	Eventos
RF-19	El sistema clasificará a los usuarios según su rol: <code>admin</code> , <code>pro</code> (profesional), <code>indv</code> (individual) y <code>press</code> (prensa).	Usuarios
RF-20	Un administrador podrá listar todos los usuarios y eliminar cuentas de forma definitiva.	Admin
RF-21	El sistema enviará notificaciones internas a los usuarios ante interacciones relevantes.	Notificaciones
RF-22	Un usuario podrá guardar búsquedas de vehículos para consultarlas posteriormente.	Búsquedas

RF-23	Los Paddocks permitirán agrupar a usuarios, anuncios y eventos por estilo de vida.	Comunidad
-------	--	-----------

2.1.2 Requisitos No Funcionales

ID	Requisito	Categoría
RNF-01	Las respuestas de la API no superarán los 300 ms en condiciones normales de carga.	Rendimiento
RNF-02	Las contraseñas se almacenarán siempre cifradas con el algoritmo bcrypt (factor de coste 12).	Seguridad
RNF-03	Todos los endpoints que modifiquen datos requerirán autenticación mediante Bearer Token (Sanctum).	Seguridad
RNF-04	Las rutas administrativas estarán protegidas adicionalmente por el middleware <code>role:admin</code> .	Seguridad
RNF-05	La aplicación deberá estar operativa en los principales navegadores modernos (Chrome, Firefox, Edge, Safari).	Compatibilidad
RNF-06	La interfaz de usuario adoptará un diseño responsive adaptado a escritorio y dispositivos móviles.	Usabilidad
RNF-07	El código del backend seguirá la arquitectura MVC y los principios SOLID .	Mantenibilidad
RNF-08	La API estará documentada con el estándar OpenAPI 3.0 y accesible vía Swagger UI.	Documentación
RNF-09	La aplicación se contenerizará con Docker para garantizar entornos reproducibles entre desarrollo y producción.	Portabilidad
RNF-10	La infraestructura de producción se definirá como código (laC) mediante Terraform sobre AWS.	Escalabilidad

RNF-11	Los listados de recursos paginados devolverán un máximo de 15 registros por página por defecto.	Rendimiento
RNF-12	Los archivos multimedia subidos por los usuarios se validarán en tipo y tamaño antes de ser almacenados.	Seguridad

2.2 Resumen de Viabilidad

IntellCar es una **startup tecnológica española del sector de la automoción** cuyo objeto social es el desarrollo y la explotación de una plataforma digital 360° para entusiastas del motor. A continuación se recoge un resumen ejecutivo del plan de empresa elaborado como parte del módulo IPE del proyecto intermodular.

Identidad y propuesta de valor

El nombre **IntellCar** —fusión de *Intelligent* y *Car*— transmite tecnología y pasión por el motor. Su eslogan es *"Todo lo que tiene motor, cobra vida."*

La propuesta de valor diferencial reside en integrar en una única plataforma cuatro funcionalidades que los competidores (Coches.net, Wallapop, motor.es) ofrecen de forma fragmentada: compraventa de vehículos, organización de eventos, red social especializada y garaje personal. IntellCar cubre el *customer journey* completo del aficionado al motor.

Segmentos de clientes

Segmento	Perfil
Particulares	Personas que compran o venden su vehículo de segunda mano
Entusiastas (<i>petrolheads</i>)	Aficionados que buscan comunidad, eventos y contenido de motor
Concesionarios y <i>dealers</i>	Profesionales que necesitan visibilidad digital
Creadores de contenido	Periodistas, <i>influencers</i> y canales especializados en motor

Modelo de negocio (fuentes de ingresos)

- **Freemium Marketplace:** anuncios básicos gratuitos; anuncios destacados de pago (posición *top*, badge *Pro Dealer*).
- **Suscripción Premium:** garaje ilimitado, estadísticas de anuncios y experiencia sin publicidad.
- **Publicidad segmentada:** banners de marcas automovilísticas dirigidos por Paddock (perfil de usuario).
- **Comisión por lead:** por cada contacto generado entre comprador y vendedor profesional.
- **Patrocinio de Kdds:** marcas patrocinan quedadas a cambio de visibilidad.

Análisis de mercado

El mercado objetivo principal es España, con aproximadamente **2 millones de transacciones de segunda mano al año** (DGT, 2024). No existe ningún competidor que integre las cuatro funcionalidades de IntellCar, lo que constituye su principal ventaja competitiva.

Fortalezas

- Plataforma integral (4 módulos)
- API REST escalable con Laravel
- Comunidad y fidelización
- Backend ya construido y documentado

Debilidades

- Sin tracción inicial de usuarios
- Equipo reducido (2 personas)
- Presupuesto de marketing limitado

Oportunidades

- Mercado de segunda mano en auge
- Tendencia a comunidades de nicho
- Monetización de eventos y creadores

Amenazas

- Competidores con grandes recursos
- Cambios regulatorios (RGPD)
- Coste elevado de captación de usuarios

Producto — los cuatro distritos

Distrito	Descripción	Monetización
----------	-------------	--------------

Marketplace	Compraventa de vehículos filtrada por Paddocks (estilos de vida)	Anuncios destacados, comisión <i>lead</i>
Eventos (Kdds)	Organización y asistencia a quedadas con geolocalización	Patrocinios de marca
El Universo	Red social de contenido motor (posts, likes, follows)	Publicidad segmentada
Mi Garaje	Vitrina personal de vehículos actuales y pasados	Suscripción Premium

Estrategia de lanzamiento (*Go-To-Market*)

- **Fase Beta (meses 1-2):** invitación a 200 *early adopters* de comunidades existentes (foros y grupos de motor).
- **Fase Crecimiento (meses 3-6):** campañas en redes sociales con contenido generado por usuarios (UGC); colaboración con *influencers* del motor en Instagram, TikTok y YouTube.
- **Fase Monetización (mes 7+):** activación de suscripciones Premium y acuerdos comerciales con *dealers*.

Forma jurídica y aspectos legales

Se constituirá como **Sociedad de Responsabilidad Limitada (S.L.)**, con capital social mínimo de 3.000 € distribuido al 50 % entre los dos socios fundadores. La marca *IntellCar* se registrará en la OEPM en las clases 42 (servicios tecnológicos) y 35 (publicidad/marketplace). La plataforma cumple con el **RGPD** y la LOPDGDD: autenticación segura con Laravel Sanctum, hash de contraseñas y rutas protegidas por *middleware*.

Estimación financiera — Año 1

Concepto	Importe
Anuncios destacados (200 uds/mes × 9,99 €)	23.976 €
Suscripciones Premium (100 uds/mes × 4,99 €)	5.988 €

Publicidad (banners)	3.000 €
Total ingresos estimados	~33.000 €
Hosting / infraestructura cloud	1.800 €
Constitución S.L. + registro de marca	600 €
Marketing digital	3.000 €
Herramientas y licencias	600 €
Total gastos estimados	~6.000 €
Resultado estimado año 1	~27.000 €

Archivo Adjunto:
[Plan de Empresa - IntellCar](#)

2.3 Casos de Uso

Se identifican tres actores principales en el sistema:

- **Usuario Anónimo:** Visitante no autenticado. Puede consultar contenido público.
- **Usuario Autenticado:** Cuenta registrada y con token válido. Puede interactuar con la plataforma.
- **Administrador:** Usuario con rol `admin`. Tiene acceso a la gestión total del contenido y usuarios.

Diagrama de Casos de Uso

El diagrama UML formal de casos de uso se incluye como figura en el documento PDF adjunto. A continuación se detalla la relación actor–caso de uso.

Acciones disponibles para el Usuario Anónimo:

- UC-01 — Registrarse
- UC-02 — Iniciar sesión
- UC-03 — Ver listado y detalle de anuncios (Marketplace)
- UC-04 — Ver feed y detalle de posts (El Universo)
- UC-05 — Ver listado y detalle de eventos (Kdds)

Acciones disponibles para el Usuario Autenticado (extiende las anteriores):

- UC-06 — Gestionar perfil (ver, editar datos y foto)
- UC-07 — Gestionar garaje virtual (añadir, editar, eliminar vehículos)
- UC-08 — Publicar y gestionar sus anuncios
- UC-09 — Publicar posts con texto y multimedia
- UC-10 — Dar like y comentar posts
- UC-11 — Seguir y dejar de seguir a otros usuarios
- UC-12 — Crear eventos (Kdds) y unirse a los existentes
- UC-13 — Cerrar sesión

Acciones exclusivas del Administrador (extiende las anteriores):

- UC-14 — Listar todos los usuarios de la plataforma
- UC-15 — Eliminar definitivamente una cuenta de usuario
- UC-16 — Eliminar definitivamente cualquier anuncio
- UC-17 — Moderar y gestionar contenido (posts, eventos)

Descripción de Casos de Uso Principales

UC-01 — Registrarse

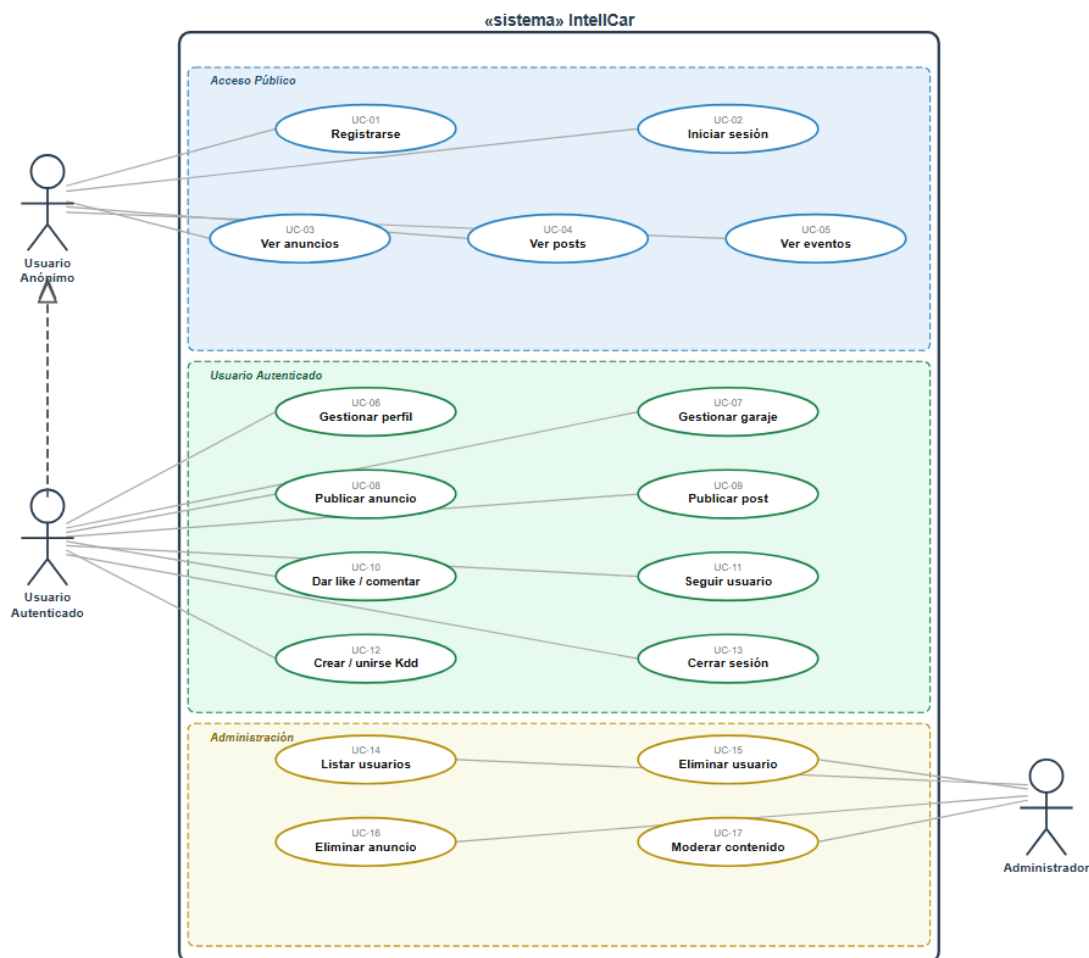
- **Actor:** Usuario Anónimo
- **Precondición:** El email y el teléfono no están registrados en el sistema.
- **Flujo principal:** El usuario completa el formulario de registro → el sistema valida los datos → crea la cuenta → devuelve el token de acceso.
- **Postcondición:** El usuario queda autenticado con un token Bearer activo.

UC-08 — Publicar Anuncio

- **Actor:** Usuario Autenticado
- **Precondición:** El usuario tiene sesión activa y dispone de los datos del vehículo (marca, modelo, motor, precio...).
- **Flujo principal:** El usuario rellena el formulario del anuncio → adjunta imágenes → envía la publicación → el sistema valida y crea el anuncio (inicialmente con `visible = false` hasta revisión).
- **Postcondición:** El anuncio aparece en el marketplace.

UC-12 — Crear / Unirse a Kdd

- **Actor:** Usuario Autenticado
- **Flujo crear:** El usuario define título, descripción, fecha, ubicación y aforo → el sistema crea el evento vinculado al creador.
- **Flujo unirse:** El usuario visualiza un evento → pulsa "Asistir" → el sistema registra su asistencia si hay plazas disponibles.



3 — Diseño de la Solución

3.1 Estructura General del Sistema y Arquitectura

IntellCar sigue una arquitectura **cliente-servidor** desacoplada con tres capas diferenciadas:

Capa	Componentes
Cliente (Browser)	Angular 19 SPA — Turborepo Monorepo (pnpm) apps/browser-client · packages/ui · packages/ts-config Comunicación: HTTP/HTTPS + Bearer Token
Backend (Laravel 12)	routes/api.php → Controllers → Models (Eloquent) → MySQL Middlewares: auth:sanctum, role:admin

	Storage: ficheros locales / Cloud Storage Docs: Swagger UI en /api/documentation
Infraestructura	Docker / Docker Compose (desarrollo y producción) Terraform → AWS(Cloud Run · Cloud SQL · Cloud Storage)

Stack Tecnológico

Capa	Tecnología	Versión
Frontend Framework	Angular	19
Frontend Language	TypeScript	~5.x
Frontend Styling	Tailwind CSS	3.x
Monorepo Manager	Turborepo + pnpm	Latest
Backend Framework	Laravel	12
Backend Language	PHP	8.3
ORM	Eloquent	(Laravel 12)
Autenticación	Laravel Sanctum	^4.0
Base de Datos	MySQL	8.x
API Docs	L5-Swagger (OpenAPI 3.0)	^11.0
Contenedores	Docker + Docker Compose	Latest
IaC	Terraform	Latest

Cloud	Amazon Web Services	—
-------	---------------------	---

Estructura del Frontend (Monorepo)

```
front/
├── apps/
│   ├── browser-client/      ← SPA Angular 19 (aplicación principal)
│   │   ├── src/app/
│   │   │   ├── core/        (guards, interceptors, servicios globales)
│   │   │   ├── features/    (módulos lazy: marketplace, social, kdds...)
│   │   │   └── shared/      (componentes reutilizables)
│   │   └── tailwind.config.js
│   └── packages/
│       ├── ui/              ← Librería de componentes compartidos
│       ├── eslint-config/
│       └── typescript-config/
```

Estructura del Backend (Laravel 12)

```
src/
├── app/
│   ├── Http/
│   │   ├── Controllers/Api/  (AuthController, MarketControl, UnivControl...)
│   │   ├── Middleware/      (RoleMiddleware)
│   │   └── Requests/         (Form Requests con validación)
│   ├── Models/              (16 modelos Eloquent)
│   └── OpenApi/              (schemas Swagger)
├── routes/
│   ├── api.php               (rutas públicas y protegidas)
│   └── auth.php
└── database/
    ├── migrations/           (28 migraciones)
    ├── factories/            (7 factories)
    └── seeders/
```

3.2 Diseño de la Base de Datos

La base de datos está compuesta por **21 tablas de dominio** más las tablas de infraestructura de Laravel (`cache`, `jobs`, `personal_access_tokens`).

Diagrama Entidad-Relación

Relaciones principales:

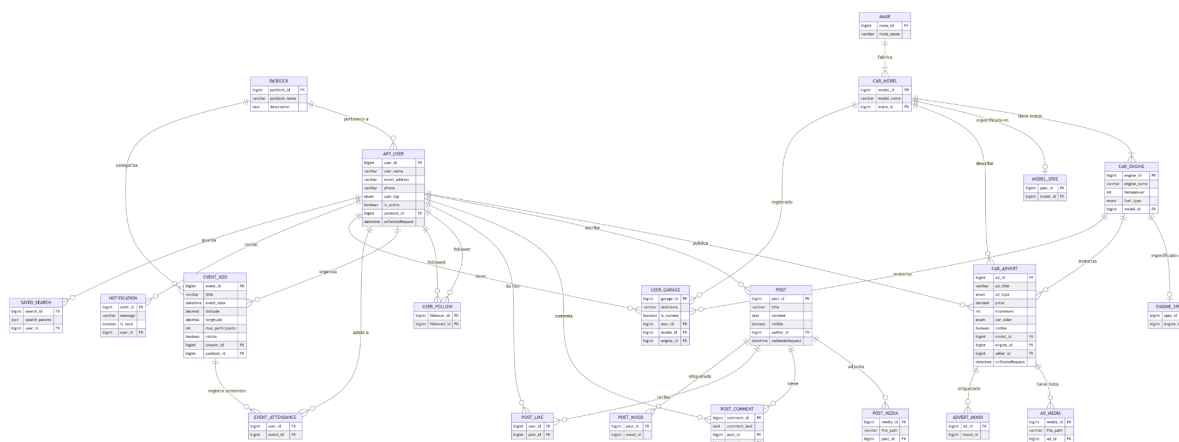
- Un **Paddock** agrupa a muchos **usuarios** y muchos **eventos**.
- Un **app_user** puede publicar muchos **anuncios**, escribir muchos **posts**, crear muchos **eventos** y tener muchos **vehículos en garaje**.
- Una **marca (make)** tiene muchos **modelos (car_model)**; un modelo tiene muchos **motores (car_engine)**.

- Un **anuncio (car_advert)** referencia un modelo, un motor y un vendedor; puede tener muchas **fotos (ad_media)**.
- Un **post** puede tener muchos **likes**, **comentarios** y **archivos multimedia**.
- Los usuarios se siguen entre sí mediante la tabla pivote **user_follow**.
- La asistencia a eventos se gestiona mediante la tabla pivote **event_attendance**.

Descripción de Tablas

Tabla	Descripción
<code>paddock</code>	Categorías de estilo de vida (clásicos, tuning, offroad...).
<code>app_user</code>	Usuarios de la plataforma. Soporta roles (admin/pro/indv/press) y soft delete.
<code>make</code>	Marcas de vehículos (Toyota, BMW, Ferrari...).
<code>car_model</code>	Modelos de vehículo, referenciados a su marca.
<code>car_engine</code>	Motorizaciones disponibles por modelo (cilindrada, CV, combustible).
<code>car_advert</code>	Anuncios del marketplace. Referencia modelo, motor y vendedor.
<code>ad_media</code>	Imágenes asociadas a un anuncio.
<code>post</code>	Publicaciones de la red social (El Universo).
<code>post_media</code>	Archivos multimedia de un post.
<code>post_like</code>	Tabla pivote N:M entre usuarios y posts (likes).
<code>post_comment</code>	Comentarios de usuarios en posts.
<code>user_follow</code>	Tabla pivote N:M de seguimiento entre usuarios.
<code>event_kdd</code>	Eventos/quedadas con soporte de geolocalización y aforo máximo.

event_attendance	Tabla pivote N:M de asistencia de usuarios a eventos.
user_garage	Vehículos del garaje virtual de cada usuario.
notification	Notificaciones internas del sistema.
saved_search	Búsquedas guardadas por usuarios, almacenadas como JSON.
model_spec	Especificaciones técnicas extendidas del modelo de vehículo.
engine_spec	Especificaciones técnicas extendidas del motor.
advert_mood	Etiquetas de estado/estilo de un anuncio (pivote).
post_mood	Etiquetas de estado/estilo de un post (pivote).



3.3 Diseño de la API

La API REST de IntelCar está organizada en **6 distritos** que agrupan los recursos por dominio funcional. La URL base es `/api/`.

La documentación interactiva completa está disponible en:

- **Desarrollo:** `http://localhost:8000/api/documentation`
- **Producción (Docker):** `https://intellcar2.duckdns.org/api/documentation`

Distrito 0 — Autenticación

Método	Endpoint	Descripción	Protección
POST	/api/auth/register	Registro de nuevo usuario	Pública
POST	/api/auth/login	Inicio de sesión → devuelve token	Pública
GET	/api/auth/me	Perfil del usuario autenticado	auth:sanctum
POST	/api/auth/logout	Invalida el token actual	auth:sanctum

Distrito 1 — Administración de Usuarios

Método	Endpoint	Descripción	Protección
GET	/api/users	Lista todos los usuarios	admin
GET	/api/users/{id}	Perfil de un usuario	auth:sanctum
DELETE	/api/users/{id}	Borrado definitivo de usuario	admin
POST	/api/users/{id}/follow	Toggle seguir / dejar de seguir	auth:sanctum

Distrito 2 — Perfil Propio y Garaje

Método	Endpoint	Descripción	Protección
GET	/api/profile	Ver datos propios	auth:sanctum
PUT	/api/profile	Editar datos propios	auth:sanctum
POST	/api/profile/picture	Actualizar foto de perfil	auth:sanctum
PATCH	/api/profile/soft-delete	Solicitar borrado de cuenta	auth:sanctum

POST	/api/profile/garage	Añadir vehículo al garaje	auth:sanctum
POST	/api/profile/garage/{id}	Editar vehículo del garaje	auth:sanctum
DELETE	/api/profile/garage/{id}	Eliminar vehículo del garaje	auth:sanctum

Distrito 3 — Market (Marketplace)

Método	Endpoint	Descripción	Protección
GET	/api/market	Listado de anuncios (con filtros)	Pública
GET	/api/market/{id}	Detalle de anuncio	Pública
PUT / PATCH	/api/market/{id}	Editar anuncio (dueño o admin)	auth:sanctum
PATCH	/api/market/{id}/soft-delete	Solicitar borrado de anuncio	auth:sanctum (dueño)
DELETE	/api/market/{id}	Borrado definitivo de anuncio	admin

Distrito 4 — Social (El Universo)

Método	Endpoint	Descripción	Protección
GET	/api/social	Feed de posts públicos	Pública
GET	/api/social/{id}	Detalle de post	Pública
POST	/api/social/{id}/like	Toggle like en un post	auth:sanctum
POST	/api/social/{id}/comment	Añadir comentario	auth:sanctum

Distrito 5 — Eventos (Kdds)

Método	Endpoint	Descripción	Protección
GET	/api/kdds	Listado de eventos	Pública
GET	/api/kdds/{id}	Detalle de evento	Pública

Convenciones de la API

- **Formato de respuesta:** JSON en todos los endpoints.
 - **Autenticación:** Cabecera `Authorization: Bearer <token>`.
 - **Paginación:** Los listados devuelven paginación con `data`, `current_page`, `last_page` y `total`.
 - **Soft Delete:** Los recursos marcados con `onDeleteRequest` o `visible = false` se excluyen automáticamente de los listados públicos.
 - **Errores:** Códigos HTTP estándar (400, 401, 403, 404, 422, 500) con mensajes descriptivos en español.
-

3.4 Identidad Visual

IntellCar adopta una identidad visual cuidada orientada a la comunidad del motor, con una estética moderna, oscura y de alto contraste que evoca la cultura del asfalto y la velocidad.

A nivel técnico, la interfaz se implementa con **Tailwind CSS** como motor de estilos, apoyado en el sistema de componentes de la librería `packages/ui` del monorepo, lo que garantiza coherencia visual en toda la aplicación y facilita la evolución del diseño de forma centralizada.

4 — Desarrollo Técnico (Core)

4.1 Backend (Laravel)

Autenticación con Sanctum.

Integración de AWS SDK (Rekognition y Comprehend).

4.2 Frontend (Turborepo)

Estructura del Monorepo (pnpm/ Turbo)

Experiencia de usuario con GSAP y Motion

4.2 Infraestructura como Código con Terraform (IaC)

Tradicionalmente, la configuración de los servidores, las redes y las bases de datos se realizaba de forma manual a través de interfaces web o comandos directos en la consola. Este enfoque suele propiciar errores humanos y dificulta la réplica exacta del entorno.

Para este proyecto, se ha optado por un enfoque moderno basado en **Infraestructura como Código (IaC)**. Esto significa que toda la estructura de servidores y servicios que necesita nuestra aplicación web no se crea a mano, sino que se define y documenta mediante archivos de texto (código).

¿Por qué Terraform?

Porque es la solución para gestionar una infraestructura “en condiciones” por las siguientes razones estratégicas:

1. **Automatización y Control del Entorno:** Terraform permite desplegar (levantar con Terraform Apply) o eliminar (destruir con Terraform Destroy) toda la infraestructura necesaria para la aplicación mediante comandos sencillos en la terminal. Esto reduce a cero los fallos de configuración manual y agiliza los tiempos de despliegue.
2. **Reutilización y Escalabilidad:** Al estar la infraestructura escrita en código, este diseño se convierte en una plantilla. Si en el futuro se necesita replicar este mismo entorno para otro proyecto, o crear un entorno idéntico de "Pruebas" (Staging) separado del de "Producción", basta con reutilizar los mismos archivos de configuración.

3. **Independencia del Proveedor (Provider Agnostic):** Terraform no está ligado a una única empresa de servicios en la nube. Permite gestionar recursos tanto en AWS, como en Google Cloud, Azure o proveedores de servidores virtuales (como DigitalOcean o Linode) utilizando el mismo lenguaje de configuración. Esto aporta flexibilidad al proyecto si se decide cambiar de proveedor en el futuro.
4. **Historial de Cambios (Estado):** La herramienta mantiene un registro exacto (archivo de estado) de cómo está el servidor en cada momento. Si realizamos un cambio en el código, Terraform sabe exactamente qué parte del servidor real debe modificar sin necesidad de borrar y reconstruir todo desde el principio.

VPC

Para garantizar el aislamiento y la seguridad de toda la infraestructura de Intellcar, no se han utilizado las redes compartidas por defecto del proveedor de la nube. En su lugar, se ha diseñado un entorno de red virtual a medida dentro de Amazon Web Services (AWS) mediante código.

A continuación, se detalla cómo se estructura lógicamente este espacio de red y los motivos técnicos de su distribución:

1. Perímetro de Red (VPC)

Se ha creado una VPC (Virtual Private Cloud) dedicada en exclusiva al proyecto, asignándole el rango de direccionamiento 10.0.0.0/16. Esto actúa como un contenedor privado y seguro en la nube. Ningún elemento externo puede comunicarse con los recursos de Intellcar a menos que se definan explícitamente las reglas de acceso y enrutamiento, lo que elimina cualquier vulnerabilidad por exposición accidental.

2. Segmentación del Espacio (Subredes)

Para organizar los diferentes recursos de la aplicación según su función y criticidad, la VPC se ha dividido en dos subredes ubicadas en zonas geográficas separadas (Zonas de Disponibilidad us-east-1a y us-east-1b). Esta separación responde a dos necesidades fundamentales: el despliegue del servidor web y la seguridad de la base de datos.

- **Subred Pública Primaria - Public Subnet 1a (10.0.1.0/24):** Ubicada en la zona de disponibilidad 1a, está configurada para permitir la interacción con el exterior. En esta subred se aloja de forma estratégica la instancia EC2 que ejecuta el backend (API en Laravel con Docker). Al ser una subred con salida directa a internet a través de la puerta de enlace (Internet Gateway), los usuarios y el frontend pueden realizar peticiones HTTP/HTTPS al servidor sin restricciones de red.
- **Subred de Respaldo y Requisito de Base de Datos - Public Subnet 1b (10.0.2.0/24):**
Esta segunda subred se ha desplegado de forma aislada en la zona de disponibilidad 1b. Su creación responde estrictamente a una medida de seguridad, alta disponibilidad y arquitectura para nuestro motor de base de datos RDS MySQL.

AWS requiere de forma obligatoria que los servicios de bases de datos gestionados (RDS) se asocien a un grupo de subredes que abarque, como mínimo, dos zonas de disponibilidad distintas. El motivo técnico es doble:

- **Seguridad y Aislamiento:** Aunque en el esquema la subred comparta el direccionamiento base, actúa como un entorno separado para el nodo secundario o de réplica de la base de datos, asegurando que la infraestructura de datos no dependa de un único punto de fallo físico.
- **Tolerancia a Fallos (Alta Disponibilidad):** Si el centro de datos de la zona 1a sufriera una caída o un problema técnico grave, AWS podría levantar o mantener la integridad de los datos utilizando los recursos de la zona 1b, garantizando que la información del proyecto Intellcar no se pierda y el sistema sea resiliente.

EC2

El núcleo computacional donde se ejecuta la lógica de negocio y el backend de Intellcar se encuentra centralizado en un servidor virtual en la nube. En lugar de utilizar un servidor físico tradicional, se ha desplegado una instancia escalable que procesa todas las peticiones del sistema.

A continuación, se describen las características técnicas de este servidor y la tecnología utilizada para desplegar la aplicación de forma eficiente:

1. Dimensionamiento del Servidor (Amazon EC2)

Para el alojamiento de la API se ha seleccionado el servicio Amazon EC2 (Elastic Compute Cloud), optando específicamente por un tipo de instancia t3.small.

La elección de este hardware virtualizado responde a un equilibrio óptimo entre coste y rendimiento para la fase actual del proyecto:

Capacidad de Cómputo: Cuenta con 2 vCPUs (procesadores virtuales) y 2 GiB de memoria RAM. Esta configuración es más que suficiente para gestionar con fluidez el servidor web, el entorno de ejecución de PHP y las peticiones concurrentes de los usuarios a la API sin sufrir degradación de rendimiento.

Arquitectura de Créditos de CPU: Al pertenecer a la familia t3, la instancia acumula créditos cuando el servidor está en reposo y los consume cuando hay picos de tráfico (por ejemplo, durante la simulación de uso por parte de los profesores o múltiples peticiones simultáneas), garantizando una alta disponibilidad económica.

2. Contenedores de Aplicación (Docker)

En lugar de instalar el servidor web, PHP y las dependencias directamente sobre el sistema operativo de la máquina virtual (lo que se conoce como una instalación "nativa"), se ha decidido aislar la API de Intellcar utilizando Docker.

La API, desarrollada en el framework Laravel, "vive" y se ejecuta por completo dentro de un contenedor Docker de forma independiente. Esta decisión arquitectónica aporta los siguientes beneficios profesionales al proyecto:

Portabilidad Absoluta: Docker empaqueta el código de la API junto con la versión exacta de PHP, las extensiones necesarias y las configuraciones del servidor web en una sola "imagen". Esto garantiza el principio de "funciona en mi ordenador y funciona en el servidor", eliminando por completo los clásicos errores de despliegue por incompatibilidad de versiones entre el entorno de desarrollo local y el entorno de producción en AWS.

Facilidad de Despliegue y Mantenimiento: Gracias a esto, actualizar la aplicación en producción es tan sencillo como descargar la nueva versión de la imagen del contenedor y reiniciarla. El proceso toma apenas unos segundos, minimizando el tiempo de inactividad de la web de Intellcar.

Aislamiento de Entornos: Al ejecutarse dentro de un entorno estanco, la aplicación no interfiere con los archivos del sistema operativo base de la EC2. Si en el futuro se quisieran añadir más microservicios o herramientas al servidor, cada una iría en su propio contenedor sin riesgo de que las dependencias de una rompan la otra.

RDS

S3

Elastic IP

Por defecto, las instancias de servidores en la nube (EC2) reciben una dirección IP pública dinámica. Esto significa que si el servidor se reinicia por mantenimiento o actualizaciones, la dirección IP cambia automáticamente, lo que rompería la conexión con cualquier dominio externo.

Para solucionar este problema, se ha reservado e implementado una IP Elástica (Elastic IP) en AWS. Esta herramienta nos proporciona una dirección IP pública estática y permanente asignada a nuestra instancia de backend. Su uso es un requisito indispensable en el proyecto por los siguientes motivos:

Enlace con el Dominio: Permite asociar de forma fija nuestro nombre de dominio público (www.intellcar.duckdns.org a través del servicio DuckDNS) hacia el servidor sin riesgo de que la web quede inaccesible.

Seguridad y Cifrado (Certbot/SSL): Al contar con una IP fija y un dominio estable, se ha podido implementar con éxito un certificado de seguridad criptográfico mediante Certbot (Let's Encrypt). Esto asegura que todo el tráfico web se realice bajo el protocolo seguro HTTPS (puerto 443), garantizando la privacidad de los datos de los usuarios y cumpliendo con las directrices de seguridad de las aplicaciones web modernas.

Security Groups

A) Grupo de Seguridad del Servidor Web (**Security Group: Web**)

Este grupo está asociado directamente a la instancia EC2 que ejecuta nuestra API y el contenedor Docker. Está diseñado para gestionar las conexiones externas de los usuarios y administradores, permitiendo la entrada únicamente a través de tres puertos específicos:

- **Puerto 22 (SSH):** Permitido exclusivamente para tareas de administración del servidor, despliegues y mantenimiento mediante la terminal.
- **Puerto 80 (HTTP):** Abierto para permitir las peticiones web iniciales de los clientes antes de ser redirigidas a la capa segura.
- **Puerto 443 (HTTPS):** El canal principal de la aplicación. Permite la entrada de tráfico cifrado para que el frontend (alojado en el Bucket S3) y los usuarios puedan consumir los servicios de la API de Intelcar de forma segura.

B) Grupo de Seguridad de la Base de Datos (**Security Group: DB**)

El almacenamiento de la información es el activo más crítico del proyecto y, por lo tanto, no debe tener ninguna exposición a internet. Este grupo de seguridad protege nuestro servicio RDS MySQL aplicando una regla de aislamiento estricta:

Puerto 3306 (MySQL): Se ha configurado una regla de entrada cerrada al mundo exterior. La base de datos solo acepta peticiones que provengan del Grupo de Seguridad Web (EC2).

De este modo, aunque un atacante intentase conectarse directamente a la base de datos desde internet, el cortafuegos bloqueará la conexión de inmediato. La única forma de interactuar con los datos de Intelcar es que la petición sea tramitada y validada previamente por nuestra propia aplicación en la EC2, blindando por completo el almacenamiento del sistema.

5 — Seguridad, Calidad y Despliegue

Moderación automática por IA

Dockerización en entornos locales en desarrollo y en AWS en producción.

Despliegue continuo (Deploy.yml con GitHub Actions)

Agregación del Dominio Duck DNS

Certificación SSL

6 — Conclusiones.

6.3 Conclusiones Técnicas

6.3 Conclusiones Personales (José)

6.3 Conclusiones Personales (Juan)

Desarrollar IntellCar ha sido, sin duda, la experiencia técnica más completa de mi formación como desarrollador web. Afrontar desde cero un proyecto full-stack real —con sus decisiones de arquitectura, sus imprevistos y sus iteraciones— me ha dado una perspectiva que ningún ejercicio aislado podría ofrecer.

En el plano técnico, el mayor aprendizaje ha venido de diseñar una **API REST escalable con Laravel 12** que soporte dominios tan distintos como un marketplace, una red social y un sistema de eventos, manteniéndola cohesionada bajo un único modelo de autenticación (Sanctum) y documentada con OpenAPI/Swagger. Aprendí que la arquitectura no es un lujo inicial: cada decisión de diseño se paga o se cobra con creces durante el desarrollo.

El trabajo con **Angular 19 en arquitectura standalone** dentro de un monorepo Turborepo me obligó a pensar en componentes reutilizables, gestión de estado y rendimiento desde el primer día. La integración con el backend mediante interceptores HTTP y guards de ruta fue uno de los puntos más desafiantes y, a la vez, más satisfactorios del proyecto.

Más allá de la técnica, este proyecto me ha enseñado a **gestionar la incertidumbre**: a priorizar funcionalidades, a tomar decisiones con información incompleta y a entregar valor incremental en lugar de perseguir la perfección. Son habilidades que, estoy convencido, marcarán mi carrera profesional desde el primer día de trabajo.

7. Futura Expansión del Proyecto

IntellCar está diseñado desde el principio para escalar. La API REST stateless, la separación entre frontend y backend, y la infraestructura en contenedores (Docker / AWS) permiten incorporar nuevas capas sin reescribir lo existente. A continuación se detallan las tres líneas de expansión planificadas para las próximas versiones.

v1.1

App móvil en React Native

Desarrollar una aplicación nativa para iOS y Android que consuma la misma API REST ya existente. React Native permite compartir lógica de negocio entre plataformas y reducir el tiempo de desarrollo. Las funcionalidades prioritarias para la app serán las de mayor uso en movilidad: **exploración del Marketplace** (con filtros por Paddock), **notificaciones push** de Kdds y actividad social, y **gestión del garaje personal** con cámara integrada para subir fotos de vehículos. La app se autenticará mediante los mismos tokens Bearer de Sanctum que usa el cliente web, sin necesidad de cambios en el backend.

v1.2

Pasarela de pago — Stripe

Integrar **Stripe** como pasarela de pago para activar los flujos de monetización definidos en el plan de empresa. Se implementarán tres productos:

- **Anuncios destacados:** pago único por anuncio (checkout de una sola vez).
- **Suscripción Premium:** cobro recurrente mensual/anual mediante Stripe Billing y webhooks para gestionar altas, bajas y renovaciones automáticamente.
- **Patrocinio de Kdds:** pago manual con Stripe Payment Links para acuerdos con marcas y organizadores de eventos.

v2.0

Versión PRO — funcionalidades avanzadas

La versión 2.0 incorporará capacidades que transforman IntellCar de plataforma de contenido en una herramienta inteligente:

- **Motor de recomendación:** algoritmo basado en el historial del garaje, los Paddocks del usuario y sus interacciones sociales para sugerir anuncios, Kdds y perfiles afines (ML con Python, expuesto como microservicio REST).
- **Chat en tiempo real:** mensajería privada entre usuarios interesados en un anuncio, implementada con **WebSockets** (Laravel Reverb o Pusher) y sincronizada con el sistema de notificaciones ya existente.
- **Panel Pro Dealer:** dashboard para concesionarios con métricas de anuncios (vistas, contactos, tasa de conversión) y herramientas de gestión de inventario en lote (importación CSV / API).

- **SEO dinámico:** generación de landing pages estáticas por marca y modelo (Angular SSR / pre-rendering) para captar tráfico orgánico.

Versión	Funcionalidad	Tecnología	Estado
v1.0	API REST completa, auth, marketplace, garaje, social, eventos	Laravel 12 + Angular 19 + MySQL	✓ Completada
v1.1	App móvil nativa	React Native + misma API	Planificada
v1.2	Pagos y suscripciones	Stripe Billing + Webhooks	Planificada
v2.0	Recomendación ML, chat en tiempo real, panel dealer, SSR	Python ML · Laravel Reverb · Angular SSR	Hoja de ruta

Anexos

Anexo I — Plan de Empresa Completo (IPE)

Documento elaborado en el módulo de **Iniciativa Emprendedora (IPE)**. Recoge el análisis completo del modelo de negocio de IntellCar: estudio de mercado detallado, análisis de la competencia, plan de marketing y comunicación, plan operativo, forma jurídica (constitución de S.L., trámites en OEPM, obligaciones fiscales) y plan financiero proyectado a 3 años. El resumen ejecutivo de dicho documento se incluye en el apartado **2.2** de esta memoria.

[Plan de Empresa - IntellCar](#)